

IEEE-CIS Fraud Detection with Autoencoders

Project Overview

This repository demonstrates a **complete MLOps workflow** for fraud detection using an autoencoder neural network, deployed on Azure and Kubernetes, with CI/CD automation via GitHub Actions. The pipeline covers data preprocessing, model training, cloud retraining, containerized deployment, and automated updates.

Quick Links

- **GitHub Repository:**

<https://github.com/LovepreetSinghGhuman/MLopsAutoencoder.git>

- **Demo Video:**

https://drive.google.com/file/d/18_wjC4EDZMhxeHaHiFTGqlicmHjhTNaA/view?usp=drive_link

Brief Explanation of the Project

- **Data:** IEEE-CIS fraud detection dataset (preprocessed as shown in notebook_original/IEEE_CIS_AutoencoderV4.ipynb).

- **Model:** Autoencoder neural network for anomaly detection.

- **Preprocessing:** Data cleaning, feature engineering, and scaling are performed in the notebook and src/train.py.

- **Pipeline:** End-to-end automation from data to deployment, using Azure ML for training and AKS for serving.

Assignment Task Mapping

Task 1: Cloud Training (Azure ML)

- **Cloud Service:** Azure ML is used for model training and retraining.
- **Automation:** Training is triggered by GitHub Actions (.github/workflows/ci-cd.yaml), using deployment/train-job.yaml.
- **Artifacts:** Model files (Keras model, scaler, config, threshold) are saved to models/ by the workflow.
- **Scripts:** Training logic is in src/train.py. Data cleaning and feature engineering are in notebook_original/IEEE_CIS_AutoencoderV4.ipynb.
- **Reproducibility:** All preprocessing and feature engineering steps are consistent between notebook, training script, and API.
- **Screenshots to include in report:**
 - Azure ML job submission and completion (from portal or CLI).
 - Pipeline run in GitHub Actions.

Azure AI | Machine Learning Studio

Hogeschool West-Vlaanderen > fraud-workspace > Models > fraud_detector_autoencoder:18

fraud_detector_autoencoder:18

Details Versions Artifacts Endpoints Jobs Data Feature sets Responsible AI

Refresh Archive Use this model Share model

Attributes

Name
fraud_detector_autoencoder

Version
18

Created on
Jun 12, 2025 2:16 AM

Created by
d3c4bb87-68ae-48e3-9b01-0a5d39b4a4d4

Type
CUSTOM

Created by job
--

Asset ID
azureml:/locations/westeurope/workspaces/833eba4d-f7a7-4e7f-b319-e109666ac7c5/models/fraud_detector_autoencoder/versions/18

Tags

No tags

Properties

azureml.datastoreid : /subscriptions/6c966d63-bdd5-4858-969d-657d3b06a354/resourceGroups/fraud-rg/providers/Microsoft.MachineLearningServices/workspaces/fraud-workspace/datastores/workspaceblobstore

Description

Click edit icon to add a description

Model catalog

Authoring

- Notebooks
- Automated ML
- Designer
- Prompt flow

Assets

- Data
- Jobs
- Components
- Pipelines
- Environments

Models

Endpoints

Manage

- Compute
- Monitoring
- Data Labeling
- Linked Services
- Connections

LovepreetSinghGhuman / MLopsAutoencoder

Code Issues Pull requests Actions Projects Wiki Security Insights Settings

Build_and_Deploy

Fix: error converter #90

Re-run all jobs

Summary

Jobs

- build-and-train
- build-and-push-frontend
- deploy

Run details

- Usage
- Workflow file

Triggered via push 24 minutes ago

LovepreetSinghGhuman pushed c2e9c2c main

Status: Success

Total duration: 12m 16s

Artifacts: --

ci-cd.yaml

on: push

```
graph LR; build-and-train[build-and-train 11m 19s] --> deploy[deploy 51s]; build-and-push-frontend[build-and-push-frontend 29s] --> deploy;
```

Task 2: Kubernetes Deployment

- **Backend:** FastAPI app (src/score.py) containerized with deployment/Dockerfile.
- **Frontend:** Responsive HTML (frontend/index.html), CSS (frontend/style.css), and JavaScript (frontend/script.js) served by NGINX (frontend/Dockerfile). The frontend is cleanly separated for maintainability and style.
- **Kubernetes Manifests:**
 - Backend: deployment/k8s/deployment.yaml, deployment/k8s/service.yaml
 - Frontend: frontend/frontend-nginx-deployment.yaml
 - Ingress: deployment/k8s/ingress.yaml routes `/predict` to FastAPI and all other paths to the static frontend.
- **Reverse Proxy:** Ingress controller is used for routing and reverse proxy.
- **User Interaction:** The frontend allows users to upload a CSV or Excel file and displays fraud predictions. The UI is styled with CSS and uses JavaScript for asynchronous file upload and result display.
- **Microservice Communication:**
 - Ingress routes `/predict` to the backend API, all other paths to the frontend.
 - [Include a diagram in your report showing this flow.]
- **Screenshots to include in report:**
 - Frontend UI (file upload and result).
 - FastAPI `/docs` page.
 - Output of `kubectl get pods,svc,ingress` to show running services.

```
PS C:\VSCode\mlopsAutoencoder> kubectl get services
NAME          TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
fraud-backend ClusterIP     10.0.247.152  <none>         8000/TCP         9h
frontend      LoadBalancer 10.0.30.178   4.175.221.18   80:30177/TCP     9h
kubernetes    ClusterIP     10.0.0.1      <none>         443/TCP         9h
```

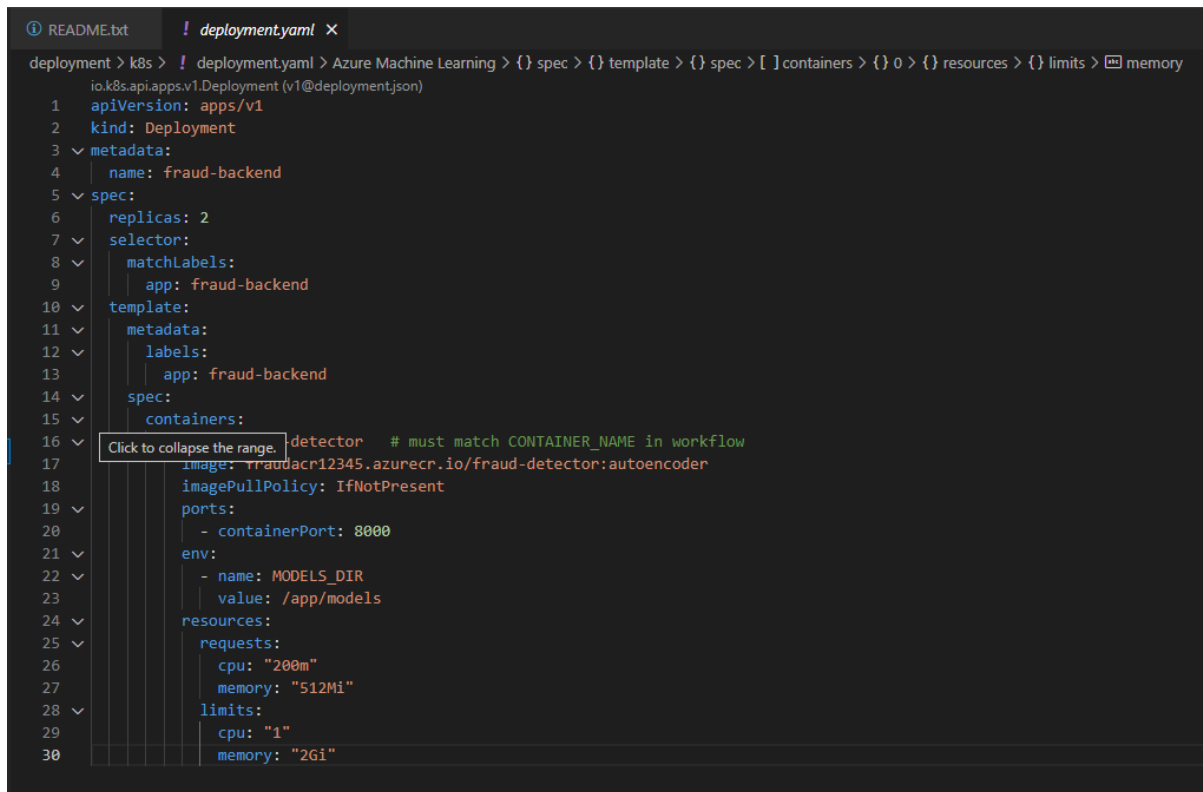
```
PS C:\VSCode\MlopsAutoencoder> kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
fraud-backend-645dd845bb-mtvwv	1/1	Running	0	66m
fraud-backend-645dd845bb-zjq1t	1/1	Running	2 (60m ago)	66m
frontend-55d4598cbb-wn57p	1/1	Running	0	9h

```
PS C:\VSCode\MlopsAutoencoder>
```

```
deployment > k8s > ! ingress.yaml > Azure Machine Learning > {} spec > [ ] rules > {} 0 > {} http > [ ] paths > {} 2 > {} backend > {} service > {} p
3 metadata:
4   name: fraud-ingress
5   annotations:
6     nginx.ingress.kubernetes.io/proxy-body-size: "10000m" # Allow up to 20MB uploads (adjust as needed)
7 spec:
8   ingressClassName: nginx
9   rules:
10    - http:
11      paths:
12        - path: /predict
13          pathType: Prefix
14          backend:
15            service:
16              name: fraud-backend
17              port:
18                number: 8000
19        - path: /health
20          pathType: Prefix
21          backend:
22            service:
23              name: fraud-backend
24              port:
25                number: 8000
26        - path: /
27          pathType: Prefix
28          backend:
29            service:
30              name: frontend
31              port:
32                number: 80
```

```
deployment > k8s > ! service.yaml > Azure Machine Learning > apiVersion
io.k8s.api.core.v1.Service (v1@service.json)
1 apiVersion: v1
2 kind: Service
3 metadata:
4   name: fraud-backend
5 spec:
6   selector:
7     app: fraud-backend
8   ports:
9     - protocol: TCP
10       port: 8000
11       targetPort: 8000
12   type: ClusterIP
```



```
deployment > k8s > ! deployment.yaml > Azure Machine Learning > {} spec > {} template > {} spec > [ ] containers > {} 0 > {} resources > {} limits > memory
io.k8s.api.apps.v1.Deployment (v1@deployment.json)
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: fraud-backend
5  spec:
6    replicas: 2
7    selector:
8      matchLabels:
9        app: fraud-backend
10   template:
11     metadata:
12       labels:
13         app: fraud-backend
14     spec:
15       containers:
16         detector: # must match CONTAINER_NAME in workflow
17         image: fraudacr12345.azurecr.io/fraud-detector:autoencoder
18         imagePullPolicy: IfNotPresent
19         ports:
20           - containerPort: 8000
21         env:
22           - name: MODELS_DIR
23             value: /app/models
24         resources:
25           requests:
26             cpu: "200m"
27             memory: "512Mi"
28           limits:
29             cpu: "1"
30             memory: "2Gi"
```

Task 3: CI/CD Automation (GitHub Actions)

- **3.1 Automatically retrain the model:**

- On every push to `main`, the workflow triggers a new Azure ML training job and waits for completion.

- **3.2 Automatically redeploy the model:**

- After training, the workflow builds and pushes a new backend Docker image to Azure Container Registry (ACR) and updates the AKS deployment.

- **3.3 Automatically redeploy the frontend/api:**

- When frontend code changes, a new Docker image is built and pushed, and the frontend deployment in AKS is updated.

- **Minimal Manual Steps:**

- After merging code to `main`, the pipeline retrains, builds, and redeploys both backend and frontend with no manual intervention.

- **Automation Details:**

- See [ci-cd.yaml](.github/workflows/ci-cd.yaml) for workflow logic.
- Model versioning is handled via Azure ML's model registry.

- All secrets are managed via GitHub Actions secrets.
- ****Screenshots to include in report:****
 - GitHub Actions workflow runs (showing retrain, build, deploy steps).
 - Any manual steps (if any) should be explained.

build-and-train
succeeded 23 minutes ago in 11m 19s

Search logs

> Set up job

3s

> Pre Log in to Azure

11s

> Checkout code

1s

> Log in to Azure

2s

> Install Azure CLI v2 and ML extension

48s

> Ensure all files are UTF-8 encoded (check, convert if needed, verify)

0s

> Submit Azure ML training job and wait for completion

7m 44s

> Download, flatten, and verify model artifacts

11s

> Register new model version in Azure ML

10s

> Build and push Docker image for Autoencoder

2m 8s

> Post Checkout code

0s

> Post Log in to Azure

0s

> Complete job

0s

build-and-push-frontend
succeeded 34 minutes ago in 29s

Search logs

> Set up job

2s

> Pre Log in to Azure

16s

> Checkout code

0s

> Log in to Azure

3s

> Log in to ACR

1s

> Build and push frontend Docker image

6s

> Debug frontend image output

0s

> Show GITHUB_OUTPUT file

0s

> Post Checkout code

0s

> Post Log in to Azure

0s

> Complete job

0s

deploy
succeeded 22 minutes ago in 51s

- > ☒ Set up job 4s
- > ☒ Pre Log in to Azure 14s
- > ☒ Checkout code 1s
- > ☒ Log in to Azure 2s
- > ☒ Install Azure CLI v2 21s
- > ☒ Set AKS context 1s
- > ☒ Make apply_all_k8s.sh executable 0s
- > ☒ Apply all Kubernetes manifests 5s
- > ☒ Update backend deployment image 0s
- > ☒ Debug frontend image in deploy job 0s
- > ☒ Update frontend deployment image 0s
- > ☒ Post Checkout code 0s
- > ☒ Post Log in to Azure 1s
- > ☒ Complete job 0s

Build_and_Deploy
☒ **Fix: error converter #90**

Re-run all jobs

Summary

Jobs

- ☒ build-and-train
- ☒ build-and-push-frontend
- ☒ deploy

Run details

- Usage
- Workflow file

Triggered via push 34 minutes ago
LovepreetSinghChuman pushed → c2e9c2c main

Status: **Success**
Total duration: **12m 16s**
Artifacts: -

ci-cd.yaml

on: push

☒ build-and-train 11m 19s

☒ build-and-push-frontend 29s

☒ deploy 51s

Repository Structure

- ****src/****: Training, inference, and pipeline scripts.
- ****models/****: Model artifacts (populated by the workflow, not tracked in git).
- ****deployment/****: Docker, conda, and Kubernetes manifests.
- ****frontend/****: Static HTML frontend and NGINX configs.
- ****notebook_original/****: Data exploration, cleaning, and pipeline prototyping.

How to Run

Cloud Workflow (Recommended)

1. **Push to main branch**

- Triggers the CI/CD pipeline automatically.
- Azure ML retrains the model, downloads artifacts, builds and pushes Docker images, and updates the Kubernetes deployment.

Manual (for local testing)

1. **Training:**

```
python src/train.py --train-data data/processed/cleaned_train.csv --output-dir models/
```

2. **Build & Deploy:**

```
docker build -f deployment/Dockerfile -t fraud-detector:latest .
```

- Push to ACR and deploy to AKS using provided manifests.

3. **Frontend:**

- Access via the service/ingress endpoint.
- Upload a CSV/Excel file and view predictions.

Architecture Diagram

...

[User] --> [Frontend (NGINX)] --> [K8s Ingress] --> [FastAPI (Autoencoder)] --> [Model Artifacts]

...

- All services are containerized and orchestrated by Kubernetes.
- Model retraining and redeployment are fully automated via CI/CD.

Reporting & Demo Guidance

- **Include links** to this repository and your demo video in your report.
- **Screenshots:**
 - Azure ML job runs, GitHub Actions pipeline, FastAPI docs, frontend UI, and Kubernetes resources.
- **Explain:**
 - How microservices communicate (see diagram above).
 - Any special Kubernetes setup or configuration.
 - Your CI/CD update strategy and why you chose it.
 - How you keep track of model versions (Azure ML registry).
- **Reflection:**
 - Briefly describe what was easy, what was difficult, and how you solved any significant problems.
- **Demo Video:**
 - Show the frontend, submit a file, get a result, and show your Kubernetes cluster is running (e.g., with `kubectl` commands).

Look into the video (google drive) for the Kubectl and upload.

Additional Notes

- All files are UTF-8 encoded (see src/convert_to_utf8.py).
 - The CI/CD pipeline checks and converts all Python, CSV, and requirements files to UTF-8 encoding before running any jobs.
 - If you see an encoding error in the pipeline, open the file in VS Code and use "Save with Encoding" → "UTF-8".
- Data files are excluded from version control via .gitignore and managed with Git LFS (.gitattributes).
- For any issues, see logs in GitHub Actions or Kubernetes pods.
- **Model artifacts in `models/` are managed by the pipeline and should not be tracked in git.**
- I have learned that working with github workflows is just plain better (easier/efficient because of the automation).
- My understanding of Microsoft Azure has improved (Model deployment, training, ...).
- Making sure all files are encoded in the same type (utf-8 or others) makes your code more robust and reduces dumb errors (workflow failing).

****Author:****

Lovepreet Singh

MLOps and AI design patterns